

Infectious Disease Simulation Final Project

Dilynn Derden

2025/08/12

1 Introduction

Infectious diseases have been a huge part of this world both in recent years and past. This project is used to simulate the outbreak and spread of an infectious disease throughout a set population. The population size, number of days sick, probability of transmission and mutation, percent of the population that is vaccinated, and the initial number of infected people can all be set going into the simulation in order to imitate different types and levels of epidemics. This will provide a clearer picture of how infectious diseases can spread throughout a population (see programming techniques described in [1]).

To perform this simulation, each individual in the determined population will be put into one of these four states after each completed day: sick, recovered, vaccinated, or susceptible. This will change day after day until the last day someone or a group of people are sick. The utilization of random interactions between individuals will help determine what state each person will be at for the next day. Overall, by modifying the population parameters this simulation will yield observations of the rate of outbreak spreading, the time the disease lasts within the population, how vaccinations prevent infections, and how mutations can increase infections.

2 File Overview

This project is organized into several C++ source files and in this writeup:

- `infect_lib.hpp`: Library that declares the `Disease`, `Person`, and `Population` classes. Full code in Appendix A.1.
- `infect_lib.cpp`: Library that implements the class behavior including the infection spread, mutation, recovery, and daily state transitions. Full code in Appendix A.2.
- `pandemic.cpp`: Contains the `main` function, collects user input, runs the simulation, and prints results and visualizations each day. Full code in Appendix A.3.
- `CMakeLists.txt`: Creates the build configuration for compiling the program.
- `project.pdf`: This written report detailing the project and results from it.

3 Model Description

This code will be completed in C++. To create an organized model for this simulation, classes will be used to separate the three different categories. These categories are:

Disease Class

Keeps track of the number of days a person is sick, the probability of a transmission between peoples random interactions, as well as the probability of mutation after someone gets an infection. The full class declaration is provided in Appendix A.1.

Person Class

Records the individuals current state that they are in. If they are sick, then it will track the type of strain of the disease and the number of days the sickness will last. The full class declaration is provided in Appendix A.1.

Population Class

Follows several objects within the Person class. This includes, the randomly assigned people that are infected and vaccinated, any contacts/interactions between individuals, applying mutations from new infections, tracking the states and mutations of people in the entire population, and advancing to the next day. The full class declaration and implementation are provided in Appendices A.1 and A.2.

This object-oriented structure follows the programming methods presented in [1]. A visualization will also be created to showcase the state that each individual is in at the end of each day. The legend is shown below:

- **s** = susceptible
- **-** = sick
- **r** = recovered
- **v** = vaccinated

4 Methods

To gauge a collection of different types of populations, the **Disease** object will be capturing the user input of the following parameters each time the simulation is ran:

- population size
- number of days sick
- probability of transmission
 - Note: On a decimal scale of 0: never spreads – 1: always spreads
- mutation probability
 - Note: On a decimal scale of 0: never mutates – 1: always mutates
- percentage vaccinated
- initial number of infected
 - Note: Assuming the user makes this value less than the population size

Real world disease parameters were gathered by surveillance summaries provided by the CDC [2] and the Ohio Department of Health [3]. Next, the **Population** object is assigned with all of the individuals starting state as susceptible. The main loop will repeat until there are no sick individuals remaining. During each day of the simulation:

1. Each person that is sick will make several random interactions with other individuals.
2. Each contact may result in an infection based on the transmission probability.
3. If an infection occurs, mutation may then occur based on mutation probability.
4. Individuals who were already sick progress towards recovering.
5. The program will then count and print the numbers of sick, susceptible, recovered, and vaccinated individuals. The state visualization will also be printed after each day has ended (see Appendix A.3).

5 Results

The simulation was run several times with a multitude of different types of parameters to observe how each parameter influenced the spread of an infectious disease. The visualization created for each day, which can be seen in Appendix A.3, helped determine which parameters affected the population in different ways.

After evaluation, the higher the transmission was led to quicker and larger disease outbreaks. It was also found that greater vaccination rates lowered the total amount of infections and shortened the length of the disease outbreak. More obviously, mutation probability affected how many new strains there were, but it did not result in an overall change in outbreak size.

Section 5.1 has three example simulations modeled after the outbreaks of COVID-19, seasonal influenza, and measles in the state of Ohio, based on statewide summaries from [2] and [3].

5.1 A Real World Example: COVID-19 vs. Influenza vs. Measles

Connecting the simulation to a real-world example, three infectious diseases, COVID-19, seasonal influenza, and measles, will be compared using disease outbreak statistics from the state of Ohio. Since Ohio has a population of 11.8 million people, all parameters were scaled down by a factor of 0.000169 to match the 2000 person population for the simulation. These statewide trends were referenced from CDC reports [2] and Ohio health summaries [3]. The goal of this example is not to reproduce exact Ohio data, but to notice how different parameter choices can affect the behavior of these specific types of infectious disease outbreaks.

Inputs	
Population size	2000
Number of days sick	7
Probability of transmission	0.55
Mutation probability	0.50
Percentage vaccinated	40%
Initial number of infected	5
Outputs	
Peak number sick in a day	1155
Total recovered	1195
Outbreak duration (days)	15
Estimated mutations	561

Table 1: Statistics for a COVID-19 outbreak in Ohio.

Inputs	
Population size	2000
Number of days sick	4
Probability of transmission	0.30
Mutation probability	0.20
Percentage vaccinated	50%
Initial number of infected	5
Outputs	
Peak number sick in a day	365
Total recovered	886
Outbreak duration (days)	22
Estimated mutations	182

Table 2: Statistics for a seasonal influenza outbreak in Ohio.

Inputs	
Population size	2000
Number of days sick	8
Probability of transmission	0.90
Mutation probability	0.05
Percentage vaccinated	20%
Initial number of infected	3
Outputs	
Peak number sick in a day	1600
Total recovered	1586
Outbreak duration (days)	13
Estimated mutations	58

Table 3: Statistics for a measles outbreak in Ohio.

6 Discussion

After running the simulation with these parameters and receiving the outputs, it is now known how different characteristics have led to the data trends we are seeing from the outbreak. Measles, having a high transmission probability, had the fastest and largest outbreak where most of the population was infected, not considering the vaccinated individuals. COVID-19 had a slower outbreak, but still had a massive peak. Finally, influenza was the smallest, but longest-lasting, outbreak because of its smaller transmission probability and high vaccination rate. These behaviors align with the real world characteristics reported in [2] and [3].

With all simulations taken into account, vaccination played a huge role in reducing the total amount of infections and slowing down the spread of a disease. It was also noticed that mutation did not significantly change outbreak size within this model. Overall, this example of comparisons demonstrates that even adjusting just a few parameters can create the general patterns observed in real-world infectious disease outbreak scenarios.

7 Conclusion

Ultimately, this simulation illustrates just how a simple C++ program can model such an important topic, like infectious diseases, and can give evidence on this pressing topic to aid or support a communities decisions. This is done through changing the models parameters such as population size, number of days someone has been sick, the probability of transmission and mutation, the communities vaccination percentage, and how many people were initially infected by the infectious disease. Afterwords, the results can aid a conclusion of the type of outbreak it is and if it resembles or is similar to any past outbreaks. The overall computational approach reflects the computational modeling techniques taught in [1]. Hopefully, the outcome of these simulations will convince populations how important it is to get vaccinated to limit or lower any infectious diseases out-breaking into their community.

References

- [1] Victor Eijkhout. *Introduction to Scientific Programming in C++*. 2025 Edition.
- [2] Centers for Disease Control and Prevention. *COVID-19, Influenza, and Measles Surveillance Summaries*. U.S. Department of Health and Human Services.
- [3] Ohio Department of Health. *Infectious Disease Annual Summary Reports*. Ohio Department of Health Publications.

APPENDIX

Appendix A: Source Code

This appendix contains the full source code files used in the infectious disease simulation.

A.1 infect_lib.hpp: Class Declarations

```
#ifndef INFECT_LIB_HPP
#define INFECT_LIB_HPP

#include <vector>
#include <random>
#include <iostream>

// Disease
//determines how long someone stays sick and how likely the infection could spread
class Disease {
private:
    int days_sick_total_;
    double spread_chance_;
    double mutation_chance_;

public:
    Disease(int days_sick_total,
            double spread_chance,
            double mutation_chance);

    int get_days_sick() const;
    double get_spread_chance() const;
    double get_mutation_chance() const;
};

// Health State
//health states a person could be in
enum class HealthState {
    Susceptible,
    Sick,
    Recovered,
    Vaccinated
};

// Person
//tracking health state and has a sick day count down
class Person {
private:
    HealthState current_state_;
    int days_remaining_sick_;
    int active_strain_id_;

public:
    Person();

    void vaccinate();
    void infect(const Disease &d, int strain_id);
};
```

```

    void one_more_day();

    bool is_sick() const;
    bool is_susceptible() const;
    bool is_recovered() const;
    bool is_vaccinated() const;

    int strain_id() const;
};

// Population
//stores all people and updates them every day
class Population {
private:
    std::vector<Person> residents_;
    int next_available_strain_id_;
    int mutation_counter_;

public:
    Population(int size);

    void vaccinate_random(double fraction,
                          std::default_random_engine &gen);

    void infect_random(const Disease &d, int count,
                      std::default_random_engine &gen);

    int count_infected() const;
    int count_vaccinated() const;
    int count_susceptible() const;
    int count_recovered() const;
    int total_mutations() const;

    void advance_one_day(const Disease &d, int contacts_per_sick,
                        std::default_random_engine &gen);

    void print_line(std::ostream &os) const;
};

#endif

```

A.2 infect_lib.cpp: Class Implementations

```

#include "infect_lib.hpp"

// Disease

Disease::Disease(int days_sick_total,
                 double spread_chance,
                 double mutation_chance)
    : days_sick_total_(days_sick_total),
      spread_chance_(spread_chance),
      mutation_chance_(mutation_chance)
{}

int Disease::get_days_sick() const {
    return days_sick_total_;
}

```

```

}

double Disease::get_spread_chance() const {
    return spread_chance_;
}

double Disease::get_mutation_chance() const {
    return mutation_chance_;
}

// Person

Person::Person()
    : current_state_(HealthState::Susceptible),
      days_remaining_sick_(0),
      active_strain_id_(-1)
{}

void Person::vaccinate() {
    current_state_ = HealthState::Vaccinated;
    days_remaining_sick_ = 0;
    active_strain_id_ = -1;
}

void Person::infect(const Disease &d, int strain_id) {
    if (current_state_ == HealthState::Susceptible) {
        current_state_ = HealthState::Sick;
        days_remaining_sick_ = d.get_days_sick();
        active_strain_id_ = strain_id;
    }
}

void Person::one_more_day() {
    if (current_state_ == HealthState::Sick) {
        if (days_remaining_sick_ > 0) {
            --days_remaining_sick_;
        }
        if (days_remaining_sick_ == 0) {
            current_state_ = HealthState::Recovered;
        }
    }
}

bool Person::is_sick() const {
    return current_state_ == HealthState::Sick;
}

bool Person::is_susceptible() const {
    return current_state_ == HealthState::Susceptible;
}

bool Person::is_recovered() const {
    return current_state_ == HealthState::Recovered;
}

bool Person::is_vaccinated() const {
    return current_state_ == HealthState::Vaccinated;
}

```

```

int Person::strain_id() const {
    return active_strain_id_;
}

// Population

Population::Population(int size)
    : residents_(size),
      next_available_strain_id_(1),
      mutation_counter_(0)
{}

//randomly vaccinate a fraction of the population
void Population::vaccinate_random(double fraction,
                                   std::default_random_engine &gen)
{
    if (fraction < 0.0) {
        fraction = 0.0;
    } else if (fraction > 1.0) {
        fraction = 1.0;
    }

    int population_size = static_cast<int>(residents_.size());
    int target_count = static_cast<int>(fraction * population_size);

    if (target_count <= 0) {
        return;
    }

    std::uniform_int_distribution<int> pick(0, population_size - 1);

    int vaccinated_so_far = 0;
    while (vaccinated_so_far < target_count) {
        int idx = pick(gen);
        if (!residents_[idx].is_vaccinated()) {
            residents_[idx].vaccinate();
            ++vaccinated_so_far;
        }
    }
}

//randomly infect a number of susceptible people
void Population::infect_random(const Disease &d, int count,
                               std::default_random_engine &gen)
{
    int population_size = static_cast<int>(residents_.size());

    int susceptible_count = 0;
    for (int i = 0; i < population_size; ++i)
        if (residents_[i].is_susceptible())
            ++susceptible_count;

    if (susceptible_count == 0) return;
    if (count > susceptible_count) count = susceptible_count;

    std::uniform_int_distribution<int> pick(0, population_size - 1);

    int infected_so_far = 0;
    while (infected_so_far < count) {

```



```

        int idx = pick(gen);
        if (residents_[idx].is_susceptible()) {
            //initial infections are base strain = 0
            residents_[idx].infect(d, 0);
            ++infected_so_far;
        }
    }
}

//count functions are utilized

int Population::count_infected() const {
    int result = 0;
    int n = static_cast<int>(residents_.size());
    for (int i = 0; i < n; ++i)
        if (residents_[i].is_sick())
            ++result;
    return result;
}

int Population::count_vaccinated() const {
    int result = 0;
    int n = static_cast<int>(residents_.size());
    for (int i = 0; i < n; ++i)
        if (residents_[i].is_vaccinated())
            ++result;
    return result;
}

int Population::count_susceptible() const {
    int result = 0;
    int n = static_cast<int>(residents_.size());
    for (int i = 0; i < n; ++i)
        if (residents_[i].is_susceptible())
            ++result;
    return result;
}

int Population::count_recovered() const {
    int result = 0;
    int n = static_cast<int>(residents_.size());
    for (int i = 0; i < n; ++i)
        if (residents_[i].is_recovered())
            ++result;
    return result;
}

int Population::total_mutations() const {
    return mutation_counter_;
}

//increase the simulation by one day
void Population::advance_one_day(const Disease &d, int contacts_per_sick,
                                std::default_random_engine &gen)
{
    int n = static_cast<int>(residents_.size());
    if (n == 0) return;

    std::vector<int> will_be_infected(n, 0);

```

```

std::vector<int> new_strain(n, -1);

std::uniform_int_distribution<int> pick(0, n - 1);
std::uniform_real_distribution<double> prob(0.0, 1.0);

//each sick person may infect random people in a day
for (int i = 0; i < n; ++i) {
    if (!residents_[i].is_sick()) continue;

    for (int c = 0; c < contacts_per_sick; ++c) {
        int j = pick(gen);
        if (j == i) continue;
        if (!residents_[j].is_susceptible()) continue;

        if (prob(gen) < d.get_spread_chance()) {

            if (will_be_infected[j] == 0) {
                will_be_infected[j] = 1;

                // Mutations
                double rm = prob(gen);
                int strain = 0;
                if (rm < d.get_mutation_chance()) {
                    strain = next_available_strain_id_;
                    ++next_available_strain_id_;
                    ++mutation_counter_;
                }
                new_strain[j] = strain;
            }
        }
    }
}

//apply infections
for (int i = 0; i < n; ++i)
    if (will_be_infected[i] == 1 && residents_[i].is_susceptible()) {
        int strain_to_use = (new_strain[i] < 0) ? 0 : new_strain[i];
        residents_[i].infect(d, strain_to_use);
    }

//update people by a day
for (int i = 0; i < n; ++i)
    residents_[i].one_more_day();
}

//visual of states
void Population::print_line(std::ostream &os) const {
    int n = static_cast<int>(residents_.size());
    for (int i = 0; i < n; ++i) {
        char c = '?';
        if (residents_[i].is_sick()) c = '-';
        else if (residents_[i].is_recovered()) c = 'r';
        else if (residents_[i].is_vaccinated()) c = 'v';
        else if (residents_[i].is_susceptible()) c = 's';
        os << c << '␣';
    }
    os << '\n';
}

```

A.3 pandemic.cpp: Main Simulation Program

```
#include "infect_lib.hpp"
#include <iostream>

int main() {

    int population_size;
    std::cout << "Population_size:_";
    std::cin >> population_size;

    int duration_days;
    std::cout << "Number_of_days_sick:_";
    std::cin >> duration_days;

    double transmission_prob;
    std::cout << "Probability_of_transmission_(Note:_On_a_decimal_scale_of_0:_never_spreads_-1:_always_spreads):_";
    std::cin >> transmission_prob;

    double mutation_prob;
    std::cout << "Mutation_probability_(Note:_On_a_decimal_scale_of_0:_never_mutates_-1:_always_mutates):_";
    std::cin >> mutation_prob;

    double percent_vaccinated;
    std::cout << "Percentage_vaccinated:_";
    std::cin >> percent_vaccinated;

    int initial_sick;
    std::cout << "Initial_number_of_infected_(Note:_Please_make_sure_the_value_is_less_than_the_Population_Size):_";
    std::cin >> initial_sick;

    int contacts_per_sick = 6;

    Disease disease(duration_days, transmission_prob, mutation_prob);
    Population pop(population_size);

    std::random_device rd;
    std::default_random_engine gen(rd());

    pop.vaccinate_random(percent_vaccinated / 100.0, gen);
    pop.infect_random(disease, initial_sick, gen);

    int day = 0;
    std::cout << "\nSIMULATION:\n\nLegend:\nSick:_\nRecovered:_r\nVaccinated:_v\nSusceptible:_s\n\n";

    while (pop.count_infected() > 0) {
        day++;
        std::cout << "\nDay_" << day
            << '\n'
            << "sick:_ " << pop.count_infected()
            << '\n'
            << "susceptible:_ " << pop.count_susceptible()
            << '\n'
            << "recovered:_ " << pop.count_recovered()
            << '\n';
    }
```

```

        << "    vaccinated: " << pop.count_vaccinated()
        << '\n';

    pop.print_line(std::cout);

    pop.advance_one_day(disease, contacts_per_sick, gen);
}

std::cout << "\n\nFINAL:\nNumber_of_days_the_disease_ended_after: " << day << '\n';
std::cout << "Total_mutations: "
        << pop.total_mutations()
        << "\n";
return 0;
}

```

Appendix B: Original Ohio Outbreak Statistics

The following values summarize and estimate representative statewide case totals for COVID-19, seasonal influenza, and measles in Ohio.

B.1 COVID-19 (Ohio)

Metric	Value
Total cases in major wave	710,000
Peak daily cases (approx.)	12,000
Estimated infected fraction of population	6%
Typical duration of statewide wave	2–3 months

B.2 Seasonal Influenza (Ohio)

Metric	Value
Annual estimated symptomatic cases	50,000–100,000
Peak hospitalized per week (approx.)	1,000
Estimated infected fraction of population	0.5%
Typical duration of flu season	3–4 months

B.3 Measles (Ohio)

Metric	Value
Recent outbreak size	85 cases
Hospitalizations (approx.)	36
Vaccination coverage (statewide)	88–92%
Typical outbreak duration	Weeks to months